

```
# =====
# Import Required Libraries
# =====

# Data Manipulation
import pandas as pd
import numpy as np

# Data Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Statistical Analysis
from scipy import stats

# Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Ignore Warnings
import warnings
warnings.filterwarnings("ignore")

# Display all columns
pd.set_option('display.max_columns', None)

# Plot Style
plt.style.use('ggplot')

print("All libraries imported successfully!")
```

All libraries imported successfully!

```
from google.colab import files
```

```
uploaded = files.upload()
```

[Choose files](#) diabetes.csv
diabetes.csv(text/csv) - 23873 bytes, last modified: 23/07/2026 - 100% done
Saving diabetes.csv to diabetes.csv

```
import pandas as pd

# Load dataset
df = pd.read_csv('diabetes.csv')

# Display first 5 rows
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome	
0	6	148	72	35	0	33.6	0.627	50	1	
1	1	85	66	29	0	26.6	0.351	31	0	
2	8	183	64	0	0	23.3	0.672	32	1	
3	1	89	66	23	94	28.1	0.167	21	0	
4	0	137	40	35	168	43.1	2.288	33	1	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.tail()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome	
763	10	101	76	48	180	32.9	0.171	63	0	
764	2	122	70	27	0	36.8	0.340	27	0	
765	5	121	72	23	112	26.2	0.245	30	0	
766	1	126	60	0	0	30.1	0.349	47	1	
767	1	93	70	31	0	30.4	0.315	23	0	

```
print("Number of Rows :", df.shape[0])
print("Number of Columns :", df.shape[1])
```

df.shape

Number of Rows : 768
Number of Columns : 9
(768, 9)

df.columns

Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
 'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
 dtype='object')

df.dtypes

	0
Pregnancies	int64
Glucose	int64
BloodPressure	int64
SkinThickness	int64
Insulin	int64
BMI	float64
DiabetesPedigreeFunction	float64
Age	int64
Outcome	int64

dtype: object

```
# Display dataset information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Pregnancies            768 non-null   int64
1   Glucose                768 non-null   int64
2   BloodPressure          768 non-null   int64
3   SkinThickness          768 non-null   int64
4   Insulin                768 non-null   int64
5   BMI                    768 non-null   float64
6   DiabetesPedigreeFunction 768 non-null   float64
7   Age                    768 non-null   int64
8   Outcome                768 non-null   int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

```
# Statistical summary
df.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	76
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	

```
df.describe(include='all')
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000

```
# Missing values
df.isnull().sum()
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

```
# Duplicate records
duplicates = df.duplicated().sum()

print("Duplicate Rows:", duplicates)
```

Duplicate Rows: 0

```
# Number of unique values
df.nunique()
```

	0
Pregnancies	17
Glucose	136
BloodPressure	47
SkinThickness	51
Insulin	186
BMI	248
DiabetesPedigreeFunction	517
Age	52
Outcome	2

dtype: int64

```
print("Rows :", df.shape[0])
print("Columns :", df.shape[1])
```

Rows : 768
Columns : 9

```
df.sample(5, random_state=42)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
668	6	98	58	33	190	34.0	0.430	43	0
324	2	112	75	32	0	35.7	0.148	21	0
624	2	108	64	0	0	30.8	0.158	21	0
690	8	107	80	0	0	24.6	0.856	34	0
473	7	136	90	0	0	29.9	0.210	50	0

```
# Count zero values in important medical columns

columns = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in columns:
    print(f"{col}: {(df[col] == 0).sum()}")
```

```
Glucose: 5
BloodPressure: 35
SkinThickness: 227
Insulin: 374
BMI: 11
```

```
import numpy as np

columns = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

df[columns] = df[columns].replace(0, np.nan)
```

```
df.isnull().sum()
```

	0
Pregnancies	0
Glucose	5
BloodPressure	35
SkinThickness	227
Insulin	374
BMI	11
DiabetesPedigreeFunction	0
Age	0
Outcome	0

```
dtype: int64
```

```
for col in columns:
    df[col].fillna(df[col].median(), inplace=True)
```

```
df.isnull().sum()
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

```
dtype: int64
```

```
df.dtypes
```

	0
Pregnancies	int64
Glucose	float64
BloodPressure	float64
SkinThickness	float64
Insulin	float64
BMI	float64
DiabetesPedigreeFunction	float64
Age	int64
Outcome	int64

dtype: object

```
df.to_csv("diabetes_cleaned.csv", index=False)
```

```
print("Cleaned dataset saved successfully!")
```

Cleaned dataset saved successfully!

```
import matplotlib.pyplot as plt
import seaborn as sns

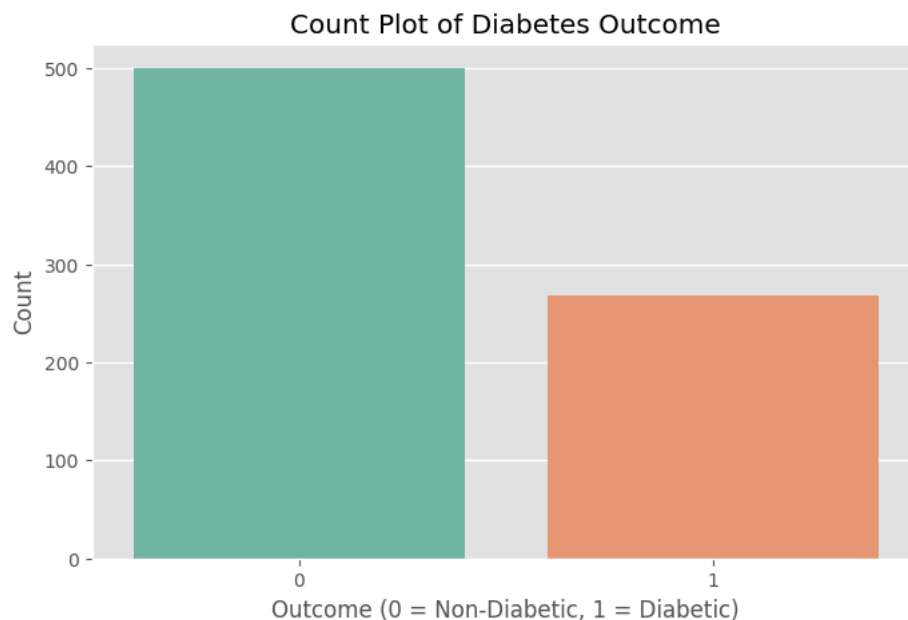
plt.figure(figsize=(8,5))

sns.countplot(x='Outcome', data=df, palette='Set2')

plt.title("Count Plot of Diabetes Outcome")
plt.xlabel("Outcome (0 = Non-Diabetic, 1 = Diabetic)")
plt.ylabel("Count")

plt.show()

# Explanation
print("Explanation:")
print("This graph shows the number of diabetic and non-diabetic patients.")
print("Most patients belong to the non-diabetic class.")
```



Explanation:
This graph shows the number of diabetic and non-diabetic patients.
Most patients belong to the non-diabetic class.

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,5))

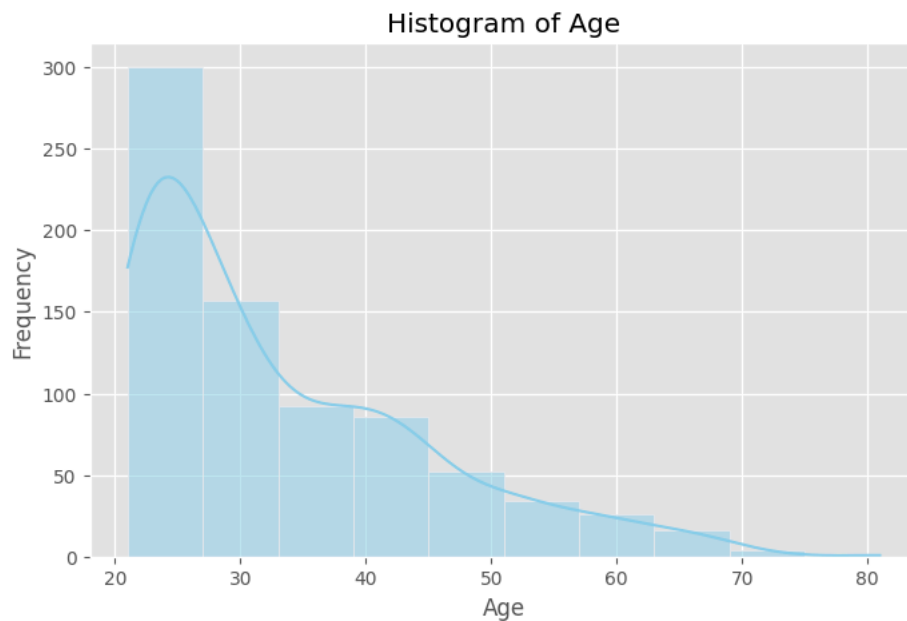
sns.histplot(df['Age'], bins=10, kde=True, color='skyblue')

plt.title("Histogram of Age")
```

```
plt.xlabel("Age")
plt.ylabel("Frequency")

plt.show()

# Explanation
print("Explanation:")
print("This graph shows the distribution of patients' ages.")
print("Most patients are between 20 and 40 years old.")
```



Explanation:
This graph shows the distribution of patients' ages.
Most patients are between 20 and 40 years old.

```
import matplotlib.pyplot as plt
import seaborn as sns

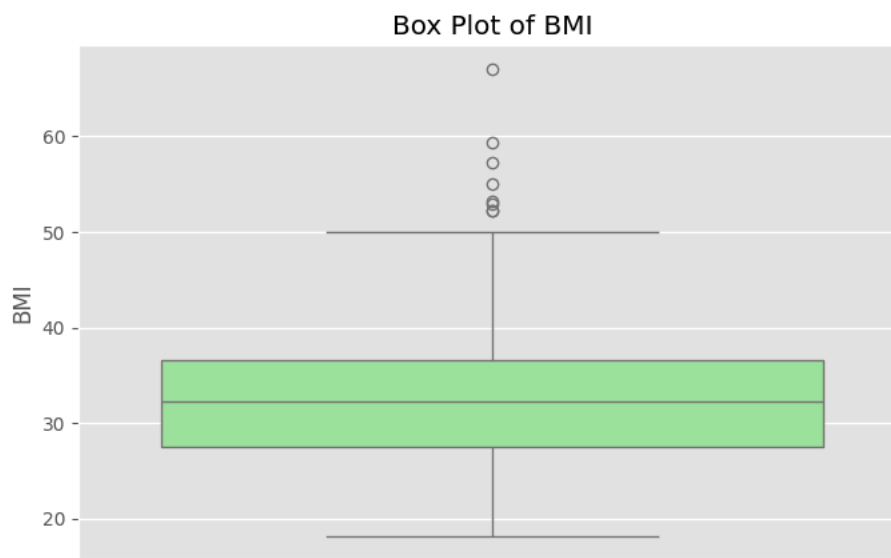
plt.figure(figsize=(8,5))

sns.boxplot(y=df['BMI'], color='lightgreen')

plt.title("Box Plot of BMI")
plt.ylabel("BMI")

plt.show()

# Explanation
print("Explanation:")
print("This graph helps identify outliers in BMI values.")
print("Some patients have unusually high BMI values.")
```



Explanation:
This graph helps identify outliers in BMI values.
Some patients have unusually high BMI values.

```
import matplotlib.pyplot as plt
import seaborn as sns

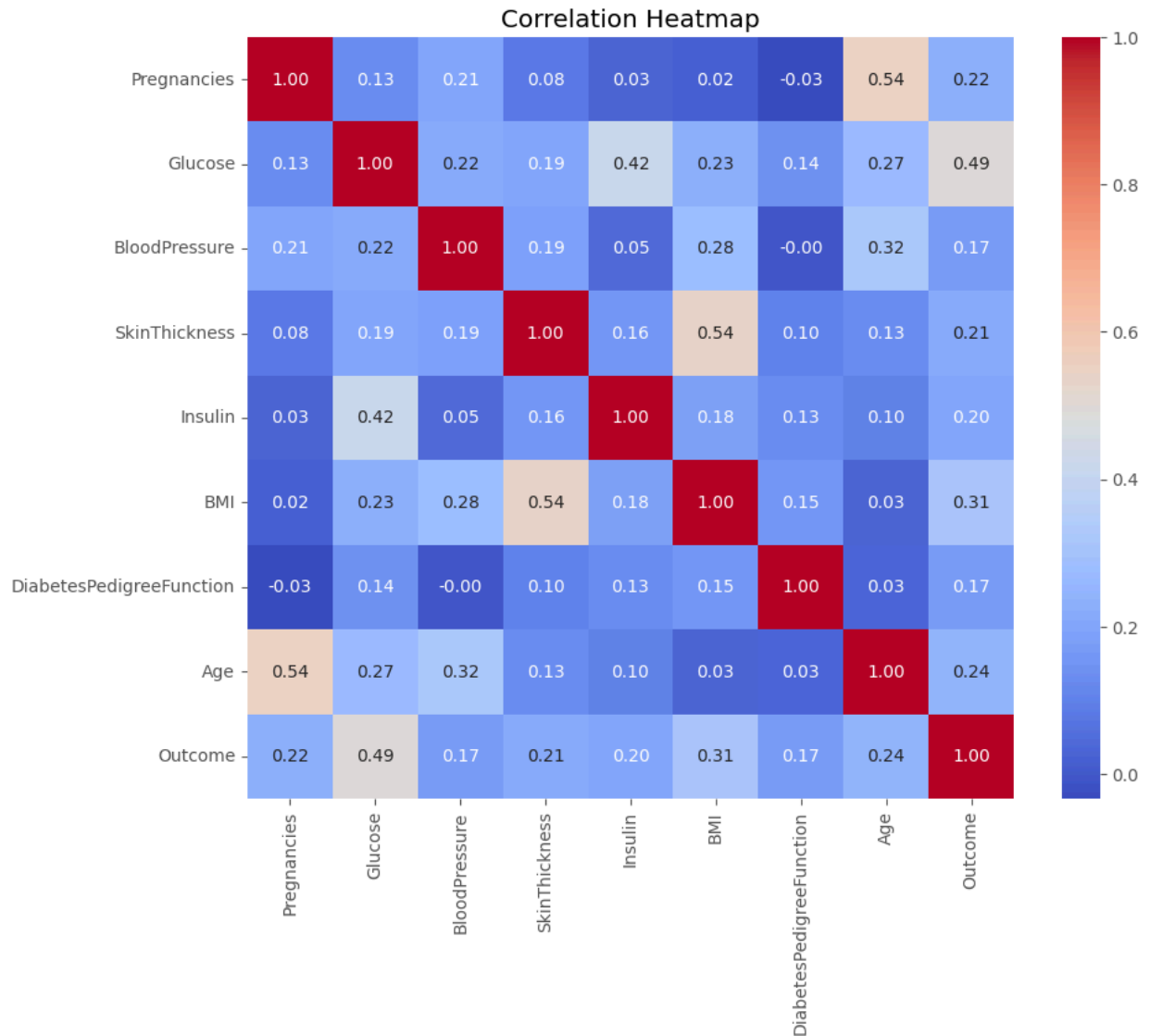
plt.figure(figsize=(10,8))

sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt=".2f")

plt.title("Correlation Heatmap")

plt.show()

# Explanation
print("Explanation:")
print("This heatmap shows the relationship between all numerical features.")
print("Darker colors indicate stronger positive or negative correlations.")
```



Explanation:
This heatmap shows the relationship between all numerical features.
Darker colors indicate stronger positive or negative correlations.

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,5))

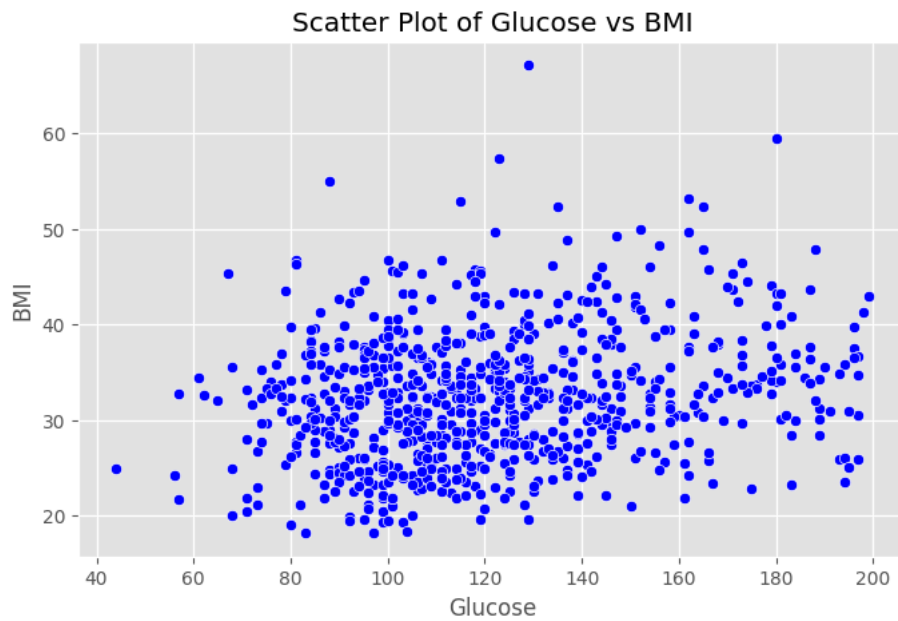
sns.scatterplot(x='Glucose', y='BMI', data=df, color='blue')

plt.title("Scatter Plot of Glucose vs BMI")
plt.xlabel("Glucose")
plt.ylabel("BMI")

plt.show()

# Explanation
print("Explanation:")
```

```
print("This graph shows the relationship between Glucose and BMI.")  
print("Higher glucose values are generally associated with higher BMI.")
```



Explanation:
This graph shows the relationship between Glucose and BMI.
Higher glucose values are generally associated with higher BMI.

```
import matplotlib.pyplot as plt  
import seaborn as sns  
  
sns.pairplot(df, hue='Outcome')  
  
plt.show()  
  
# Explanation  
print("Explanation:")  
print("This graph shows the relationship between all numerical features.")  
print("It helps identify patterns and differences between diabetic and non-diabetic patients.")
```